

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ
ФАКУЛЬТЕТ ИНФОКОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ

Отчет по лабораторной работе №1
по курсу «Алгоритмы и структуры данных»
Тема: «Сортировка вставками, выбором, пузырьковая»
Вариант 19

Выполнила:

Сигаева К.Л.

К3140

Проверил:

Афанасьев А.В.

Санкт-Петербург

2024 г.

Содержание отчета

Содержание отчета	2
Задачи по варианту	3
Задача №1. Сортировка вставкой	3
Задача №5. Сортировка выбором	6
Задача №7. Знакомство с жителями Сортлэнда	9
Дополнительные задачи	12
Задача №4. Линейный поиск	12
Задача №6. Пузырьковая сортировка	15
Вывод	18

Задача №1. Сортировка вставкой

Текст задачи

1 задача. Сортировка вставкой

Используя код процедуры Insertion-sort, напишите программу и проверьте сортировку массива $A = \{31, 41, 59, 26, 41, 58\}$.

- **Формат входного файла (input.txt).** В первой строке входного файла содержится число n ($1 \leq n \leq 10^3$) — число элементов в массиве. Во второй строке находятся n различных целых чисел, по модулю не превосходящих 10^9 .
- **Формат выходного файла (output.txt).** Одна строка выходного файла с отсортированным массивом. Между любыми двумя числами должен стоять ровно один пробел.
- Ограничение по времени. 2сек.
- Ограничение по памяти. 256 мб.

Выберите любой набор данных, подходящих по формату, и протестируйте алгоритм.

Листинг кода

```
import time
import tracemalloc

start_time = time.perf_counter()
tracemalloc.start()

with open("input.txt") as file:
    numbers = list(map(int, file.readlines()[1].split()))

for i in range(1, len(numbers)):
    temp = numbers[i]
    j = i - 1
    while j >= 0 and temp < numbers[j]:
        numbers[j + 1] = numbers[j]
        j = j - 1
    numbers[j + 1] = temp

with open("output.txt", "w") as file:
    file.write(" ".join(map(str, numbers)))

print(f"Время работы: {time.perf_counter() - start_time} c")
print(f"Память: {tracemalloc.get_traced_memory()[1] / 2 ** 20} Мб")
```

Текстовое объяснение решения

1. Импортируем модуль time и tracemalloc, чтобы отслеживать время работы программы и задействованную память
2. Записываем стартовое значение времени и вызываем функцию tracemalloc.start(), чтобы начать трассировку выделения памяти
3. Открываем файл и считываем из него данные (числа)
4. Сортируем вставками:

- запускаем цикл for, который пробегает по элементам списка, начиная с первого ($i = 1$)
 - записываем текущее значение в temp
 - вводим переменную j ($j = i - 1$)
 - переходим в цикл while с условием, что j неотрицательно и текущий элемент списка меньше элемента с индексом j (т.е. он стоит на неправильном месте, т.к. элементы с индексом j стоят до него):
 - записываем предыдущий элемент с большим значением на место текущего, чтобы был восстановлен правильный порядок
 - уменьшаем значение j на 1, чтобы перейти к пред-предыдущему элементу
 - записываем в temp значение элемента с индексом j + 1
3. Открываем файл и записываем в него отсортированные числа через пробел
4. Выводим время работы кода и выделенную память

Результат работы кода на примерах из текста задачи

```

1.py  input.txt  output.txt
1    6
2    31 41 59 26 41 58
3

```

```

1.py  input.txt  output.txt
1    26 31 41 41 58 59
2

```

Результат работы кода на максимальных и минимальных значениях

```

1.py  input.txt  output.txt
1    6
2    26 31 41 41 58 59
3
4

```

```

1.py  input.txt  output.txt
1    26 31 41 41 58 59

```

```

1.py  input.txt  output.txt
1    6
2    59 58 41 41 31 26

```

```

1.py  input.txt  output.txt
1    26 31 41 41 58 59

```

	Время выполнения (с)	Затраты памяти (Мб)
Нижняя граница диапазона значений входных данных из текста задачи	0.0005372000159695745	0.017333984375
Пример из задачи	0.0005790000432170928	0.017333984375
Верхняя граница диапазона значений входных данных из текста задачи	0.0006073000258766115	0.017333984375

Вывод по задаче

В результате выполнения задачи была произведена сортировка вставками. Также было вычислено время, которое она выполняется, и выделенная память. Затраты памяти не изменялись, время увеличивалось в зависимости от количества перестановок.

Задача №5. Сортировка выбором

Текст задачи

5 задача. Сортировка выбором.

Рассмотрим сортировку элементов массива, которая выполняется следующим образом. Сначала определяется наименьший элемент массива, который ставится на место элемента $A[1]$. Затем производится поиск второго наименьшего элемента массива A , который ставится на место элемента $A[2]$. Этот процесс продолжается для первых $n - 1$ элементов массива A .

Напишите код этого алгоритма, также известного как сортировка выбором (selection sort). Определите время сортировки выбором в наихудшем случае и в среднем случае и сравните его со временем сортировки вставкой.

Формат входного и выходного файла и ограничения - как в задаче 1.

Листинг кода

```
import time
import tracemalloc

start_time = time.perf_counter()
tracemalloc.start()

with open("input.txt") as file:
    numbers = list(map(int, file.readlines()[1].split()))

for i in range(len(numbers) - 1):
    mini = i
    for j in range(i + 1, len(numbers)):
        if numbers[j] < numbers[mini]:
            mini = j
    numbers[i], numbers[mini] = numbers[mini], numbers[i]

with open("output.txt", "w") as file:
    file.write(" ".join(map(str, numbers)))

print(f"Время работы: {time.perf_counter() - start_time} c")
print(f"Память: {tracemalloc.get_traced_memory()[1] / 2 ** 20} МБ")
```

Текстовое объяснение решения

1. Импортируем модуль `time` и `tracemalloc`, чтобы отслеживать время работы программы и задействованную память
2. Записываем стартовое значение времени и вызываем функцию `tracemalloc.start()`, чтобы начать трассировку выделения памяти
3. Открываем файл и считываем из него данные (числа)
4. Сортируем выбором:
 - запускаем цикл `for`, который пробегает по элементам списка до `len(numbers) - 1`
 - записываем индекс текущего элемента в `mini`, считая, что он пока что минимален

- запускаем вложенный цикл for, который пробегает по всем элементам, начиная с того, у которого индекс $i + 1$
- если находится элемент меньше минимального, то записываем его индекс в `mini`
- после вложенного цикла меняем местами текущий элемент и минимальный

5. Открываем файл и записываем в него отсортированные числа через пробел

6. Выводим время работы кода и выделенную память

Результат работы кода на примерах из текста задачи

```
5.py  input.txt x  output.txt x
1 6
2 31 41 59 26 41 58
3
```

```
5.py  input.txt  output.txt x
1 26 31 41 41 58 59
```

Результат работы кода на максимальных и минимальных значениях

```
5.py  input.txt x  output.txt
1 6
2 26 31 41 41 58 59
```

```
5.py  input.txt  output.txt x
1 26 31 41 41 58 59
```

```
5.py  input.txt x  output.txt
1 6
2 59 58 41 41 31 26
```

```
5.py  input.txt x  output.txt
1 6
2 31 41 59 26 41 58
3
```

	Время выполнения (с)	Затраты памяти (Мб)
Нижняя граница диапазона значений входных данных из текста задачи	0.0005545000312849879	0.017363548278808594
Пример из задачи	0.0006010999786667526	0.017333984375
Верхняя граница диапазона значений	0.0006491999956779182	0.017363548278808594

входных данных из текста задачи		
------------------------------------	--	--

Вывод по задаче

В результате выполнения задачи была произведена сортировка выбором. Также было вычислено время, которое она выполняется, и выделенная память. Затраты памяти практически не изменялись, а время увеличивалось в зависимости от неотсортированности списка.

Задача №7. Знакомство с жителями Сортлэнда

Текст задачи

7 задача. Знакомство с жителями Сортлэнда

Владелец графства Сортлэнд, граф Баблсортер, решил познакомиться со своими подданными. Число жителей в графстве нечетно и составляет n , где n может быть достаточно велико, поэтому граф решил ограничиться знакомством с тремя представителями народонаселения: с самым бедным жителем, с жителем, обладающим средним достатком, и с самым богатым жителем.

Согласно традициям Сортлэнда, считается, что житель обладает средним достатком, если при сортировке жителей по сумме денежных сбережений он оказывается ровно посередине. Известно, что каждый житель графства имеет уникальный идентификационный номер, значение которого расположено в границах от единицы до n . Информация о размере денежных накоплений жителей хранится в массиве M таким образом, что сумма денежных накоплений жителя, обладающего идентификационным номером i , содержится в ячейке $M[i]$. Помогите секретарю графа мистеру Свопу вычислить идентификационные номера жителей, которые будут приглашены на встречу с графом.

- **Формат входного файла (input.txt).** Первая строка входного файла содержит число жителей n ($3 \leq n \leq 9999$, n нечетно). Вторая строка содержит описание массива M , состоящее из положительных вещественных чисел, разделенных пробелами. Гарантируется, что все элементы массива M различны, а их значения имеют точность не более двух знаков после запятой и не превышают 10^6 .
- **Формат выходного файла (output.txt).** В выходной файл выведите три целых положительных числа, разделенных пробелами — идентификационные номера беднейшего, среднего и самого богатого жителей Сортлэнда.

- Пример:

input.txt	output.txt
5 10.00 8.70 0.01 5.00 3.00	3 4 1

Если отсортировать жителей по их достатку, получится следующий массив:

$[0.01, 3][3.00, 5][5.00, 4][8.70, 2][10.00, 1]$

Здесь каждый житель указан в квадратных скобках, первое число — его достаток, второе число — его идентификационный номер. Таким образом, самый бедный житель имеет номер 3, самый богатый — номер 1, а средний — номер 4.

- Ограничение по времени. 2 сек.
- Ограничение по памяти. 256 мб.

Листинг кода

```
import time
import tracemalloc

start_time = time.perf_counter()
tracemalloc.start()

with open("input.txt") as file:
    numbers = list(map(float, file.readlines()[1].split()))

for i in range(len(numbers)):
    numbers[i] = [i + 1, numbers[i]]

for i in range(1, len(numbers)):
    temp = numbers[i]
    j = i - 1
    while j >= 0 and temp[1] < numbers[j][1]:
        numbers[j + 1] = numbers[j]
        j -= 1
    numbers[j + 1] = temp

the_poorest = numbers[0][0]
average = numbers[len(numbers) // 2][0]
the_richest = numbers[-1][0]

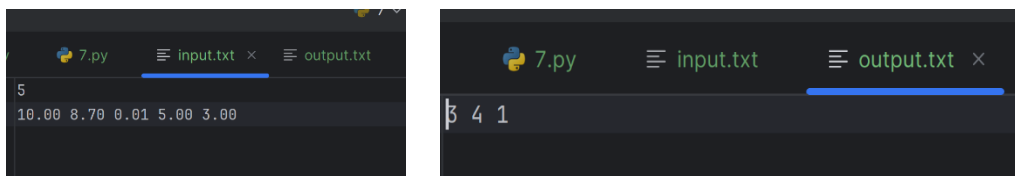
with open("output.txt", "w") as file:
    file.write(" ".join([str(the_poorest), str(average), str(the_richest)]))

print(f"Время работы: {time.perf_counter() - start_time} с")
print(f"Память: {tracemalloc.get_traced_memory()[1] / 2 ** 20} МБ")
```

Текстовое объяснение решения

1. Импортируем модуль time и tracemalloc, чтобы отслеживать время работы программы и задействованную память
2. Записываем стартовое значение времени и вызываем функцию tracemalloc.start(), чтобы начать трассировку выделения памяти
3. Открываем файл и считываем из него данные (числа)
4. Превращаем список в двумерный: каждый элемент содержит число и его индекс, начиная с 1
5. Сортируем вставками:
 - не забываем, что это двумерный список, поэтому берем элемент с индексом 1 из каждого вложенного списка (значение) при сравнении
6. Записываем в отдельные переменные индекс самого бедного, среднего и самого богатого
7. Открываем файл и записываем в него значения
8. Выводим время работы кода и выделенную память

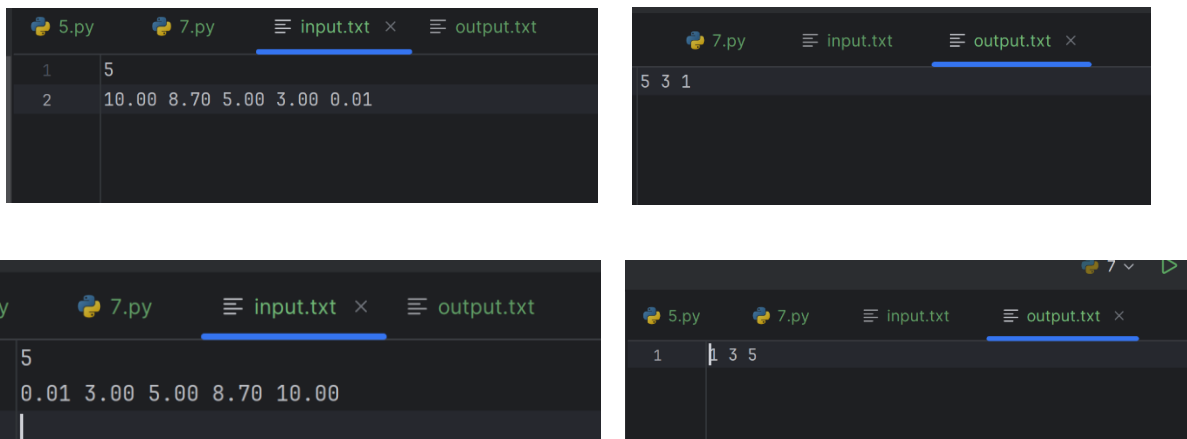
Результат работы кода на примерах из текста задачи



```
7.py input.txt output.txt
5
10.00 8.70 0.01 5.00 3.00

7.py input.txt output.txt
5 4 1
```

Результат работы кода на максимальных и минимальных значениях



```
5.py 7.py input.txt output.txt
1 5
2 10.00 8.70 5.00 3.00 0.01

7.py input.txt output.txt
5 3 1

7.py input.txt output.txt
5
0.01 3.00 5.00 8.70 10.00

5.py 7.py input.txt output.txt
1 3 5
```

	Время выполнения (с)	Затраты памяти (Мб)
Нижняя граница диапазона значений входных данных из текста задачи	0.0005163000314496458	0.01734161376953125
Пример из задачи	0.0005507000023499131	0.017371177673339844
Верхняя граница диапазона значений входных данных из текста задачи	0.0007620000396855175	0.017371177673339844

Вывод по задаче

В результате выполнения задачи была произведена сортировка жителем по достатку методом вставок. Также было вычислено время, которое она выполняется, и выделенная память. Затраты памяти практически не изменялись, а время увеличивалось в зависимости от того, какое количество перестановок надо было совершить.

Задача №4. Линейный поиск

Текст задачи

4 задача. Линейный поиск

Рассмотрим задачу поиска.

- **Формат входного файла.** Последовательность из n чисел $A = a_1, a_2, \dots, a_n$ в первой строке, числа разделены пробелом, и значение V во второй строке. Ограничения: $0 \leq n \leq 10^3$, $-10^3 \leq a_i, V \leq 10^3$
- **Формат выходного файла.** Одно число - индекс i , такой, что $V = A[i]$, или значение -1 , если V в отсутствует.
- Напишите код линейного поиска, при работе которого выполняется сканирование последовательности в поисках значения V .
- Если число встречается несколько раз, то выведите, сколько раз встречается число и все индексы i через запятую.
- Дополнительно: попробуйте найти свинью, как в лекции. Используйте во входном файле последовательность слов из лекции, и найдите соответствующий индекс.

Листинг кода

```
import time
import tracemalloc

def linear_search(numbers):
    for i in range(len(numbers)):
        if numbers[i] == x:
            return i + 1
    return -1

start_time = time.perf_counter()
tracemalloc.start()

with open("input.txt") as file:
    data = list(map(int, file.readline().split()))
    x = int(file.readline())

with open("output.txt", "w") as file:
    file.write(str(linear_search(data)))

print(f"Время работы: {time.perf_counter() - start_time} c")
print(f"Память: {tracemalloc.get_traced_memory()[1] / 2 ** 20} МБ")
```

Текстовое объяснение решения

1. Импортируем модуль `time` и `tracemalloc`, чтобы отслеживать время работы программы и задействованную память
2. Реализуем функцию линейного поиска:

- бежим по всем элементам списка и сравниваем их со значением, которое ищем
 - если значения равны, то возвращаем его индекс (считая от 1)
 - если значение не найдено, возвращаем -1
3. Записываем стартовое значение времени и вызываем функцию `tracemalloc.start()`, чтобы начать трассировку выделения памяти
 4. Открываем файл и считываем из него данные и значение, которое надо найти
 5. Открываем файл вывода и записываем в него результат работы функции
 6. Выводим время работы кода и выделенную память

Результат работы кода на примере

```

4.py  input.txt x  output.txt
1 56 8 5 34 9 2 8 7 50
2 34
3

```

```

4.py  input.txt  output.txt x
1 4

```

Результат работы кода на максимальных и минимальных значениях

```

4.py  input.txt x  output.txt
1 56 8 5 34 9 2 8 7 50
2 39
3

```

```

4.py  input.txt  output.txt x
1 -1

```

```

4.py  input.txt x  output.txt
1 56 8 5 34 9 2 8 7 50
2 56
3

```

```

4.py  input.txt  output.txt x
1 1

```

	Время выполнения (с)	Затраты памяти (Мб)
Нижняя граница	0.0004365999484434724	0.017160415649414062

диапазона значений входных данных из текста задачи		
Пример из задачи	0.00048699998296797276	0.017160415649414062
Верхняя граница диапазона значений входных данных из текста задачи	0.0006392999785020947	0.017160415649414062

Вывод по задаче

В результате выполнения задачи был написан алгоритм линейного поиска. Также было вычислено время, которое он выполняется, и выделенная память. Затраты памяти не изменялись, а время увеличивалось в зависимости от того, где находился нужный элемент.

Задача №6. Пузырьковая сортировка

Текст задачи

6 задача. Пузырьковая сортировка

Пузырьковая сортировка представляет собой популярный, но не очень эффективный алгоритм сортировки. В его основе лежит многократная перестановка соседних элементов, нарушающих порядок сортировки. Вот псевдокод этой сортировки:

```
Bubble_Sort(A):  
  for i = 1 to A.length - 1  
    for j = A.length downto i+1  
      if A[j] < A[j-1]  
        поменять A[j] и A[j-1] местами
```

Напишите код на Python и докажите корректность пузырьковой сортировки. Для доказательства корректности процедуры вам необходимо доказать, что она завершается и что $A'[1] \leq A'[2] \leq \dots \leq A'[n]$, где A' - выход процедуры Bubble_Sort, а n - длина массива A .

Определите время пузырьковой сортировки в наихудшем случае и в среднем случае и сравните его со временем сортировки вставкой.

Формат входного и выходного файла и ограничения - как в задаче 1.

Листинг кода

```
import time  
import tracemalloc  
  
def bubble_sort(numbers):  
    swapped = True  
    while swapped:  
        swapped = False  
        for i in range(len(numbers) - 1):  
            if numbers[i + 1] < numbers[i]:  
                numbers[i + 1], numbers[i] = numbers[i], numbers[i + 1]  
                swapped = True  
    return numbers  
  
start_time = time.perf_counter()  
tracemalloc.start()  
  
with open("input.txt") as file:  
    data = list(map(int, file.readlines()[1].split()))  
  
with open("output.txt", "w") as file:  
    file.write(" ".join(map(str, bubble_sort(data))))  
  
print(f"Время работы: {time.perf_counter() - start_time} c")  
print(f"Память: {tracemalloc.get_traced_memory()[1] / 2 ** 20} МБ")
```

Текстовое объяснение решения

1. Импортируем модуль time и tracemalloc, чтобы отслеживать время работы программы и задействованную память

2. Создаем функцию, которая будем осуществлять пузырьковую сортировку
 - создаем флаг перестановок
 - инициализируем цикл while с условием верности флага
 - делаем флаг ложным, т.к. перестановок в эту итерацию еще не было
 - пробегаемся во вложенном цикле for по всем парам элементов и если есть неупорядоченная, то меняем элементы в ней местами
 - если произошла перестановка, флаг истинен
3. Записываем стартовое значение времени и вызываем функцию `tracemalloc.start()`, чтобы начать трассировку выделения памяти
4. Открываем файл и считываем из него данные
5. Открываем файл вывода и записываем в него результат работы функции через пробел
6. Выводим время работы кода и выделенную память

Результат работы кода на примере

```

1 6
2 31 41 59 26 41 58
3

```

```

1 26 31 41 41 58 59

```

Результат работы кода на максимальных и минимальных значениях

```

1 6
2 26 31 41 41 58 59
3

```

```

1 26 31 41 41 58 59

```

```

1 6
2 59 58 41 41 31 26
3

```

```

1 26 31 41 41 58 59

```

	Время выполнения (с)	Затраты памяти (Мб)
Нижняя граница	0.0005840000230818987	0.017333984375

диапазона значений входных данных из текста задачи		
Пример из задачи	0.004131699970457703	0.017333984375
Верхняя граница диапазона значений входных данных из текста задачи	0.004482100019231439	0.017333984375

Вывод по задаче

В результате выполнения задачи был написан алгоритм сортировки пузырьком. Также было вычислено время, которое он выполняется, и выделенная память. Затраты памяти не изменялись, а время увеличивалось в зависимости от того, в каком порядке расположены элементы.

Вывод

- Во время выполнения этой лабораторной работы мною были написаны сортировки пузырьком, выбором, вставками и линейный поиск
- Также было вычислено время работы алгоритмов и затраты на память
- Было выяснено, что алгоритм сортировки пузырьком менее эффективен, чем алгоритм сортировки вставками, так как с теми же входными данными время работы больше не менее, чем в 10 раз